



休眠唤醒问题分析指导手册

文档版本
发布日期

V1.0
2021-04-28

版权所有 © 紫光展锐科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

紫光展锐科技有限公司



前言

概述

本文档详细介绍了手机 SoC 功耗调试的常见问题和相关分析方法。

读者对象

本文档主要适用于采用 Andorid10.0 的 OS 系统，基于 T610 UNISOC 智能机平台，从事低功耗相关软件研发与测试人员。

缩略语

缩略语	英文全名	中文解释
AP	Application Processor	应用处理器
SoC	System On Chip	芯片级系统
CPU	Central Processing Unit	中央处理器
PMIC	Power management integrated circuit	电源管理芯片

符号约定

在本文中可能出现下列标志，它所代表的含义如下。

符号	说明
 说明	用于突出重要/关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。

变更信息

文档版本	发布日期	作者	修改说明
V1.0	2021-04-28	UNISOC	初稿。

关键字

Suspend/Resume、功耗调试

目 录

1 概述	7
2 休眠唤醒问题调试手段	8
2.1 子系统不睡问题	8
2.1.1 子系统睡眠状态	8
2.1.2 未休眠模块定位	10
2.2 kernel 层资源锁问题	11
2.3 睡眠被中断唤醒问题	13
3 常见功耗问题分析方法	17
3.1 Suspend\Resume 流程常见关键字	17
3.2 无法进入 Sleep 类问题分析	20
3.2.1 Suspend 流程被打回	20
3.2.2 Suspend 流程未被执行	21
3.3 异常唤醒分析	22
3.3.1 GPIO、EIC 中断唤醒	22
3.3.2 其他中断唤醒	23
3.4 睡眠后底电流偏高问题分析	23
3.4.1 判断系统进入深睡状态	23
3.4.2 系统未进入深睡时功耗问题分析	23
3.4.3 系统已进入深睡时功耗问题分析	24

图目录

图 2-1 AP 唤醒中断架构.....	15
----------------------	----

表目录

表 2-1 电源域状态寄存器说明	8
表 2-2 各子系统睡眠状态寄存器说明	9
表 2-3 AP 中影响睡眠的模块使能位	10
表 2-4 dst 对应子系统关系	12
表 2-5 常见通道对应功能	12
表 2-6 当前具有唤醒功能的中断及其中断号	13

1 概述

在整个手机项目的量产节点中，功耗验收是一个关键指标。深睡场景下的待机功耗决定了手机的最长待机时间。而深睡场景下睡眠唤醒的问题多种多样，无法睡眠、睡眠后底电流过高、反复唤醒等问题，都会影响待机功耗。

本文以平台 AP 侧的 debug log 为核心，介绍了手机 SoC 的休眠唤醒上的常见问题和功耗调测方法。

2 休眠唤醒问题调试手段

2.1 子系统不睡问题

2.1.1 子系统睡眠状态

平台方案，当芯片内某个子系统进入深睡状态时，该子系统对应的电源域会下电；当芯片内部所有的子系统都进入深睡状态时，整个系统将进入深睡状态。

对于某个子系统不睡的问题，首先需要确认该子系统是否工作正常，低功耗相关的软件流程是否走完。如开机后发生 WCN Assert 的场景，WCN 已无法正常工作，自然不会睡眠。

对于子系统的睡眠状态，可以在 debug log 中查看。当 AP 通过 Suspend/Resume 流程进入深睡，在所有驱动的 suspend 回调执行完毕，secondary cpu 拔核之前，会打印当前系统所有的子系统的睡眠状态的寄存器，相关 log 如下：

```
[ 738.326726] c7  ##--PMU_APB.PWR_STATUS0_DBG: 0x07000700 打印子系统睡眠状态 0:上电 7:下电
[ 738.326728] c7  ##--PD_AP_SYS_STATE: 0 未关闭的子系统会单独打印出来，此时 AP 未睡
[ 738.326729] c7  ##--PD_CDMA_SYS_STATE: 0 CDMA 位于 AP 上，随 AP 关闭
[ 738.326736] c7  ##--PMU_APB.PWR_STATUS1_DBG: 0x07070707 子系统电源域状态寄存器
[ 738.326742] c7  ##--PMU_APB.PWR_STATUS2_DBG: 0x07070707
[ 738.326749] c7  ##--PMU_APB.PWR_STATUS3_DBG: 0x00070707
[ 738.326755] c7  ##--PMU_APB.PWR_STATUS4_DBG: 0x07000707
```

说明

蓝色字体为对 log 内容的解释说明，下同。

各子系统电源域状态参见表 2-1，每个 PWR_STATUSx_DBG(x:0-4)寄存器最多记录 4 个电源域状态，00 表示电源域处于上电状态，07 表示电源域处于下电状态。

Debug log 中将自动读取各个子系统电源域的关闭状态，若为打开状态，则会额外打印一行 log，标注该电源域的名称。

表2-1 电源域状态寄存器说明

寄存器	对应子系统电源域名称			
	bit31-24	bit23-16	bit15-8	bit7-0

寄存器	对应子系统电源域名称			
PWR_STATUS0_DBG	VDSP	CDMA	AP_VSP	AP
PWR_STATUS1_DBG	WTLCP_HU3GE_B	WTLCP_LDSP	WTLCP_TGDSP	WTLCP_HU3GE_A
PWR_STATUS2_DBG	WTLCP_TD_PROC	WTLCP_LTE_PROC	PUBCP	WTLCP
PWR_STATUS3_DBG		MM	GPU	AUDCP_AUDDSP
PWR_STATUS4_DBG	WTLCP_LTE_DPFEC	WTLCP_LTE_CE	PUB	AUDCP

说明

所示寄存器具体含义，各个项目不同，本文仅以 T610 为例，其他项目请参考该项目 spec，下同。

此外，对于各子系统睡眠状态，可以参考如下 log 信息。

```
[ 131.945893] c0  ##--PMU_APB.SLEEP_CTRL: 0x1000806e
```

在 SLEEP_CTRL 寄存器中，bit0-6 某 bit 为 1，说明该子系统休眠。各 bit 位与子系统的关系如下：

表2-2 各子系统睡眠状态寄存器说明

Bit	对应子系统睡眠状态
SLEEP_CTRL	
0	AP
1	WTLCP
2	PUBCP
4	CDMA
5	PUB
6	SP

说明

未标注 bit 位功能为 revert 或有其他功能，无需关注具体数值，下同。

由于 SP 子系统不下电，无法通过电源域的下电状态来判断是否休眠，但是 SP 仍能通过进入休眠降低自己的功耗。此时就可以通过该寄存器的 bit6 来判断 SP 是否正常进入休眠。

2.1.2 未休眠模块定位

当子系统低功耗相关的代码运行完毕，但是对应的电源域没有正常下电的场景，可能是该子系统上某些模块未正常关闭。如 AP 进入深睡依赖 display 模块正常关闭，未正常关闭会导致 AP 无法进入深睡。针对这种情况，debug log 中会打印 AP 上各外设模块的使能位，可以根据该寄存器信息，判断相应外设是否关闭。相关 log 如下：

```
[ 208.695359][11-24 03:12:24.695] ##--AP AHB.AHB_EB: 0x00000080
[ 208.695359][11-24 03:12:24.695] ##--AP APB.APB_EB: 0x04028000 此时串口仍在工作，uart1 使能为正常现象。
```

同样的，debug log 中将会检查影响 AP 睡眠的使能位，并单独打印出来。详细信息如下表。

表2-3 AP 中影响睡眠的模块使能位

Bit	对应模块使能位
AHB_EB	
0	DSI_EB
1	DISPC_EB
2	VSP_EB
4	DMA_ENABLE
5	DMA_EB
6	IPI_EB
APB_EB	
0	SIM0_EB
1	IIS0_EB
2	IIS1_EB
3	IIS2_EB
5	SPI0_EB
6	SPI1_EB
7	SPI2_EB
8	SPI3_EB
9	I2C0_EB
10	I2C1_EB
11	I2C2_EB
12	I2C3_EB

Bit	对应模块使能位
13	I2C4_EB
14	UART0_EB
15	UART1_EB
16	UART2_EB
22	SDIO0_EB
23	SDIO1_EB
24	SDIO2_EB
25	EMMC_EB
30	CE_SEC_EB
31	CE_PUB_EB

2.2 kernel 层资源锁问题

当系统持有资源锁(wake lock), 或存在唤醒源(wakeup sources)时, 系统将无法进入 Suspend/Resume 流程, 若已处在 Suspend/Resume 流程之中, 将会中止当前的流程, 并恢复之前 Suspend 的各项资源。如果系统持续存在唤醒源, 最终将导致系统无法进入深睡状态, 待机功耗偏高。

系统中常见的唤醒源如下:

1 musb-sprd

Usb 唤醒源。建立 USB 连接后, 该唤醒源持续 Pending。

若未插入 USB 线时, 该唤醒源 pending, 则需检查硬件上是否有外挂 usb hub 芯片, 或检查 usb 驱动是否正常工作。

2 smsg-x-x

此唤醒源为 Sipc 模块创建的唤醒源, 该模块用于和其他子系统进行通信。

当与其他子系统进行数据交互, 如 sensor hub 发送 log 信息给 AP 时, 该唤醒源会 Pending。

其唤醒系统时, 常带有相关 log:

```
irq read smsg: dst=5, channel=6, type=4, flag=0x0001, value=0x00000000
```

其中，**smmsg-x-x** 中第一个 **x** 为 **Dst**，代表相关通信的对象，对照关系如下：

表2-4 dst 对应子系统关系

值	对应的子系统
0	Application Processor(AP)
1	mini AP processor(MINIAP)
2	WCDMA processor(WCDMA)
3	Wireless Connectivity(WCN)
4	Gps processor(gnss)
5	Protocol stack processor(PSCP)
6	Power management processor(SP)
7	New Radio PHY processor
8	MODEM v3 PHY processor

smmsg-x-x 中第二个 **x** 为 **Channel**，代表通信的通道，有子通道的情况，其 **Value** 值为子通道的编号，常见的通道功能对照关系如下：

表2-5 常见通道对应功能

Channel	Value	功能
4	8	Refnotify 相关，AP/MODEM 间传递信息，如时戳等。
5		log 通道
6	0-5	AT 命令
6	22	AGPS 通道
7-9		网络通道
10-17		audio 相关
18-20		网络通道
21		diag 通道

说明

表 2-5 仅为常见通道说明，仅供参考。

常见例子：**smmsg-6-5**

其中 **6** 表示 Power management processor，来自 **SP** 侧的通信；**5** 表示 **log** 通道。

即存在 **SP** 侧的 **log** 信息，可以检查 **ylog** 是否关闭。

2.3 睡眠被中断唤醒问题

对于 UNISOC 平台，在系统进入睡眠，一个唤醒中断产生准备唤醒系统时，该中断信号将会送到位于 AON 上的 INTC，若这个中断的唤醒功能打开，则 PMU 将控制 AP 上电。

以 T610 为例，目前具有唤醒功能的中断如下：

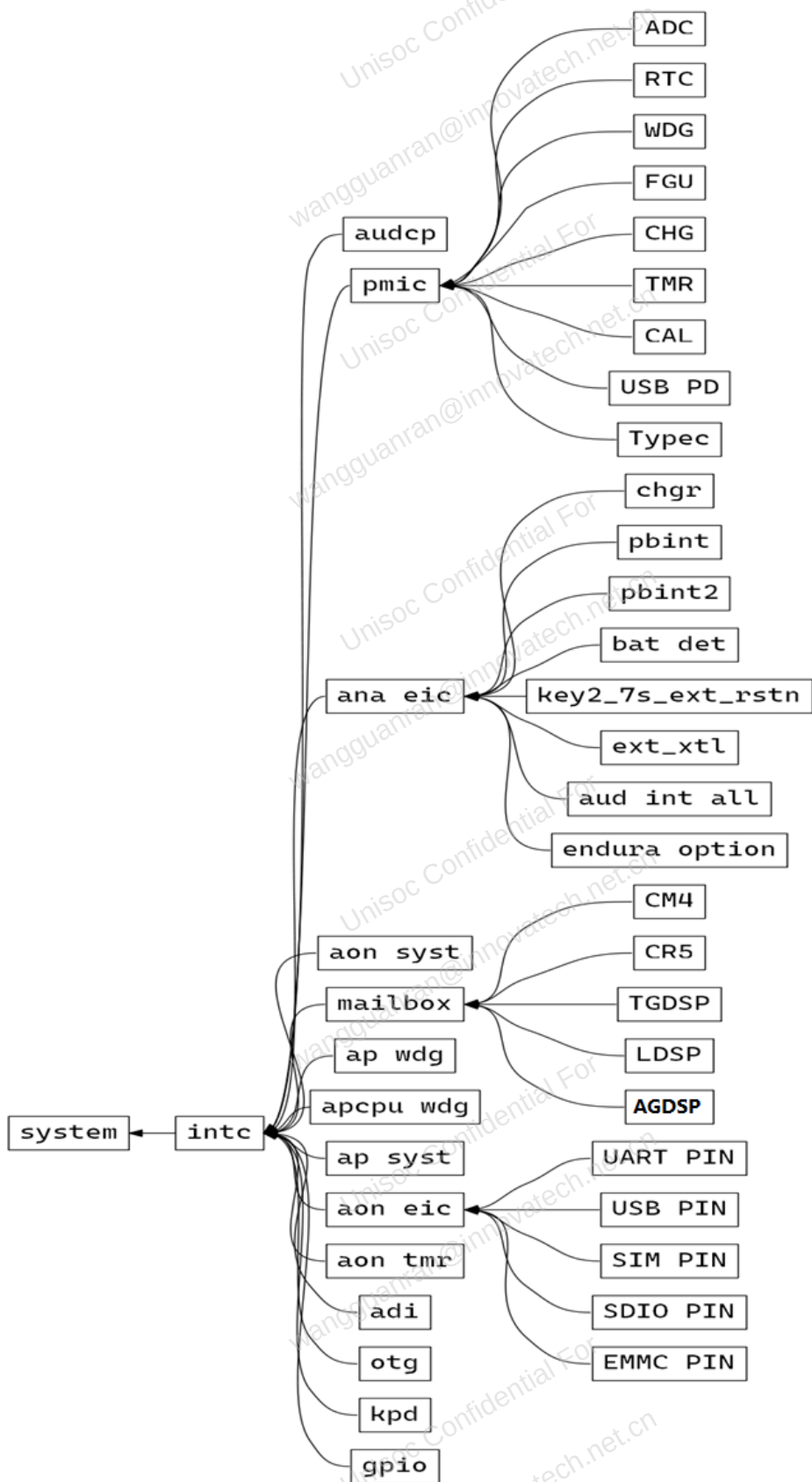
表2-6 当前具有唤醒功能的中断及其中断号

中断号	中断名称
180	AUDCP_CHN_START_CHN3
188	EIC_NON_LAT
173	ANA
104	AON_SYST
94	SEC_GPIO
93	SEC_TMR
92	SEC_RTC
91	SEC_WDG
90	SEC_EIC_NON_LAT
89	SEC_EIC
83	MBOX_TAR_AP
82	MBOX_SRC_AP
80	AP_WDG
79	APCPU_WDG
78	AP_SYST
72	EIC
71	AON_TMR
70	ADI
69	OTG
67	KPD
61	GPIO

由于存在某个模块可以产生多个中断，但是只有一条信号线连接到 INTC 上的情况，因此系统中唤醒源的数量与 INTC 上中断的数量不是完全对应的。如每个 GPIO 口都能单独的的产生中断，但是最终只会以一条中断线的形式送给 INTC。对于 INTC 上的 ANA、EIC、GPIO、Mailbox 中断，存在多个来源。

如下为 T610 的唤醒中断架构图。

图2-1 AP 唤醒中断架构



对于多级嵌套的中断，在 **debug log** 中，将逐级打印其中断信息。

以常见的 **RTC** 唤醒系统为例，**PMIC** 上 **RTC** 唤醒系统时，将打印如下 **log**：

```
[ 249.570986] ###--WAKEUP SOURCE
[ 249.597752] ###--ANA (RTC_INT)
[ 249.597754] ###--END
```

对于 **gpio**、**eic** 中断，也可以通过 **wake up by** 关键字搜索，相关 **log** 如下：

```
[ 151.341432] c0 sprd-pmic-eic sc27xx-eic: wake up by pmic eic:Power Key id:1
```

3 常见功耗问题分析方法

3.1 Suspend\Resume 流程常见关键字

NO	关键字	关键字说明
Kernel Log 中关键字		
1	suspend entry	系统开始走 suspend 流程
2	Syncing filesystems	系统休眠过程中同步文件系统，一般同步完成后会打印 done.
3	Freezing user space processes	Suspend 过程中冻结进程，完成会打印 done.
4	Suspending console	Suspend 过程中关闭 console 控制台，此时无法在输入 log
5	Disabling non-boot CPUs	Suspend 过程中拔核
6	###--AP DEEP SLEEP CONDITION CHECK	打印低功耗相关的寄存器信息
7	###--POWER DOMAIN STATE	打印电源域信息，各 sys 关闭状态
8	Enabling non-boot CPUs	系统唤醒，Resume 过程中插核
9	Restarting tasks	Resume 过程中恢复冻结的 task,完成会打印 done.
10	wake up by	对于 eic gpio 等中断唤醒，将通过该关键字打印唤醒源
11	###--WAKEUP SOURCE	唤醒系统待机的中断名称（rtc、ana、mbox、gpio 等）
12	suspend exit	系统退出 suspend，走 resume 流程
Suspend 失败的关键字		

NO	关键字	关键字说明
1	Freezing of tasks failed after	冻结进程失败，可能是冻结时有新唤醒源 Pending，或某些进程工作异常。
2	PM: dirty and writeback data is	文件系统回写数据过多，需要关闭 log
3	PM: Pending Wakeup Sources:	Suspend 期间有新的唤醒源 Pending，结合 log 看是哪个唤醒源
4	PM: Device xxx failed to suspend: error:	某个驱动 Suspend 失败，需要结合当前场景该驱动的业务判断是否正常。

正常 Suspend 流程 的 log 如下：

```
[ 131.611616] c0 PM: suspend entry (deep)
[ 131.615437] c0 PM: Syncing filesystems ... done.
[ 131.664320] c6 Freezing user space processes ... 冻结用户空间进程
[ 131.728650] c6 (elapsed 0.064 seconds) done.
[ 131.732837] c6 OOM killer disabled.
[ 131.736293] c6 Freezing remaining freezable tasks ... (elapsed 0.001 seconds) done.
[ 131.745723] c6 Suspending console(s) (use no_console_suspend to debug)
[ 131.754884] c6 sc27xx-fgu sc27xx-fgu: resume data->suspend_start_time = 131 各驱动开始执行 suspend 回调
[ 131.756272] c6 Suspend alarm: 2021-4-19 19:55:22 alarm 将于该时间唤醒系统
[ 131.785644] c6 Disabling non-boot CPUs ...
[ 131.803652] c0 CPU1: shutdown
[ 131.803665] c0 psci: CPU1 killed. CPU1 关闭
[ 131.826975] c7 CPU2: shutdown
[ 131.826980] c7 psci: CPU2 killed. CPU2 关闭
[ 131.850403] c0 CPU3: shutdown
[ 131.850415] c0 psci: CPU3 killed. CPU3 关闭
[ 131.874075] c6 CPU4: shutdown
[ 131.874080] c6 psci: CPU4 killed. CPU4 关闭
[ 131.897725] c0 CPU5: shutdown
[ 131.897737] c0 psci: CPU5 killed. CPU5 关闭
[ 131.920604] c0 CPU6: shutdown
[ 131.920615] c0 psci: CPU6 killed. CPU6 关闭
[ 131.944519] c0 CPU7: shutdown
[ 131.944530] c0 psci: CPU7 killed. CPU7 关闭
[ 131.945893] c0 ###--AP DEEP SLEEP CONDITION CHECK AP 功耗相关寄存器打印
[ 131.945893] c0 ##--AP_AHB.AHB_EB: 0x00000080
[ 131.945893] c0 ##--AP_AHB.AP_SYS_FORCE_SLEEP_CFG: 0x00000000
[ 131.945893] c0 ##--AP_APB.APB_EB: 0x04028000
[ 131.945893] c0 ##--AP_APB.APB_MISC_CTRL: 0x00000000
[ 131.945893] c0 ##--AON_APB.SP_CFG_BUS: 0x00000041
[ 131.945893] c0 ##--PMU_APB.APCPU_MODE_STATE0: 0x00070702
[ 131.945893] c0 ##--APCPU_CORE0_LOW_POWER_STATE: 2
[ 131.945893] c0 ##--PMU_APB.APCPU_MODE_STATE_FIG: 0x07070707
[ 131.945893] c0 ##--PMU_APB.APCPU_MODE_STATE1: 0x00000207
[ 131.945893] c0 ##--APCPU_CORINTH_LOW_POWER_STATE: 2
```

```

[ 131.945893] c0 ##--PMU_APB.APCPU_PWR_STATE0: 0x07070000 打印电源域相关信息
[ 131.945893] c0 ##--PD_APCPU_TOP_STATE: 0
[ 131.945893] c0 ##--PD_APCPU_C0_STATE: 0
[ 131.945893] c0 ##--PMU_APB.APCPU_PWR_STATE_FIG: 0x07070707
[ 131.945893] c0 ##--PMU_APB.APCPU_PWR_STATE1: 0x00000007
[ 131.945893] c0 ##--PMU_APB.PWR_STATUS0_DBG: 0x07000700
[ 131.945893] c0 ##--PD_AP_SYS_STATE: 0
[ 131.945893] c0 ##--PD_CDMA_SYS_STATE: 0
[ 131.945893] c0 ##--PMU_APB.PWR_STATUS1_DBG: 0x07070707
[ 131.945893] c0 ##--PMU_APB.PWR_STATUS2_DBG: 0x07070707
[ 131.945893] c0 ##--PMU_APB.PWR_STATUS3_DBG: 0x00070707
[ 131.945893] c0 ##--PMU_APB.PWR_STATUS4_DBG: 0x07000707
[ 131.945893] c0 ##--PD_PUB_SYS_STATE: 0
[ 131.945893] c0 ##--PMU_APB.SLEEP_CTRL: 0x1000806e 打印子系统睡眠信息
[ 131.945893] c0 ##--AP_DEEP_SLEEP: 0
[ 131.945893] c0 ##--PUB_SYS_DEEP_SLEEP: 0
[ 131.945893] c0 ##--PMU_APB.LIGHT_SLEEP_MON: 0x0000002f
[ 131.945893] c0 ##--AP_LIGHT_SLEEP: 0
[ 131.945893] c0 ###--END
[ 151.341432] c0 sprd-pmic-eic sc27xx-eic: wake up by pmic eic:Power Key id:1 由 eic 唤醒, 唤醒源自 power key
[ 151.341683] c0 Enabling non-boot CPUs ... 开始对各个 cpu 走上电操作
[ 151.342373] c1 CPU1: Booted secondary processor [411fd050]
[ 151.344271] c0 CPU1 is up
[ 151.345069] c2 CPU2: Booted secondary processor [411fd050]
[ 151.347100] c0 CPU2 is up
[ 151.347771] c3 CPU3: Booted secondary processor [411fd050]
[ 151.350050] c0 CPU3 is up
[ 151.350723] c4 CPU4: Booted secondary processor [411fd050]
[ 151.353256] c0 CPU4 is up
[ 151.353905] c5 CPU5: Booted secondary processor [411fd050]
[ 151.357181] c0 CPU5 is up
[ 151.357695] c6 CPU6: Booted secondary processor [413fd0a1]
[ 151.360340] c0 CPU6 is up
[ 151.360841] c7 CPU7: Booted secondary processor [413fd0a1]
[ 151.362008] c0 CPU7 is up
[ 151.546639] c0 sc27xx-fgu sc27xx-fgu: suspend->resume period time = 20, cur_time = 151 驱动开始执行 resume 回调
[ 153.610835] c6 OOM killer enabled.
[ 153.614207] c6 Restarting tasks ... done. 各进程解冻完成
[ 153.621072] c6 ###--WAKEUP SOURCE
[ 153.624391] c6 ##--ANA(EIC_INT.PBINT) 打印唤醒中断
[ 153.628184] c6 ###--END
[ 153.630687] c6 gnss pm notify event:4
[ 153.634334] c6 sprd-sensor sprd-sensorhub: status=2
[ 153.639197] c6 [ASoC: PCM ] sprd pcm pm notifier, PM POST SUSPEND.
[ 153.645322] c6 [ASoC: PCM ] sprd pcm pm notifier, PM POST SUSPEND.
[ 153.651477] c6 sprd-freqlink: resume done!
[ 153.655570] c6 PM: suspend exit
  
```

此外, 无法进入 Suspend/Resume 流程时, 将会以 30s 周期打印各电源域的状态, debug log 如下:

```

[ 127.964503] c7 ###--POWER DOMAIN STATE
[ 127.970259] c7 ##--PMU_APB.APCPU_MODE_STATE0: 0x00070702
  
```



```
[ 127.981744] c7  #--APCPU_CORE0_LOW_POWER_STATE: 2
[ 127.981750] c7  ##--PMU_APB.APCPU_MODE_STATE_FIG: 0x07020707
[ 127.992646] c7  #--APCPU_CORE5_LOW_POWER_STATE: 2
[ 127.992652] c7  ##--PMU_APB.APCPU_MODE_STATE1: 0x00000202
[ 128.005282] c7  #--APCPU_CORE7_LOW_POWER_STATE: 2
[ 128.005284] c7  #--APCPU_CORINTH_LOW_POWER_STATE: 2
[ 128.005288] c7  ##--PMU_APB.APCPU_PWR_STATE0: 0x07070000
[ 128.015580] c7  #--PD_APCPU_TOP_STATE: 0
[ 128.015582] c7  #--PD_APCPU_C0_STATE: 0
[ 128.015586] c7  ##--PMU_APB.APCPU_PWR_STATE_FIG: 0x07070007
[ 128.027436] c7  #--PD_APCPU_C5_STATE: 0
[ 128.027441] c7  ##--PMU_APB.APCPU_PWR_STATE1: 0x00000000
[ 128.037821] c7  #--PD_APCPU_C7_STATE: 0
[ 128.037827] c7  ##--PMU_APB.PWR_STATUS0_DBG: 0x07000700
[ 128.037828] c7  #--PD_AP_SYS_STATE: 0
[ 128.037829] c7  #--PD_CDMA_SYS_STATE: 0
[ 128.037835] c7  ##--PMU_APB.PWR_STATUS1_DBG: 0x07070707
[ 128.047520] c7  ##--PMU_APB.PWR_STATUS2_DBG: 0x07070707
[ 128.057126] c7  ##--PMU_APB.PWR_STATUS3_DBG: 0x00070700
[ 128.066639] c7  #--PD_AUDCP_DSP_STATE: 0
[ 128.066646] c7  ##--PMU_APB.PWR_STATUS4_DBG: 0x00000707
[ 128.077021] c7  #--PD_PUB_SYS_STATE: 0
[ 128.077022] c7  #--PD_AUDCP_SYS_STATE: 0
[ 128.077028] c7  ##--PMU_APB.SLEEP_CTRL: 0x1000804e
[ 128.086283] c7  #--AP_DEEP_SLEEP: 0
[ 128.086285] c7  #--PUB_SYS_DEEP_SLEEP: 0
[ 128.086288] c7  #--AUDCP_DEEP_SLEEP: 0
[ 128.094093] c7  ##--PMU_APB.LIGHT_SLEEP_MON: 0x0000002d
[ 128.107401] c7  #--AUDCP_LIGHT_SLEEP: 0
[ 128.107405] c7  #--AP_LIGHT_SLEEP: 0
[ 128.119518] c7  ###--END
```

说明

注：以上 log 中，加粗部分为添加的 power debug log，其余部分为 kernel 原生 log。

3.2 无法进入 Sleep 类问题分析

3.2.1 Suspend 流程被打回

系统未持锁，log 中能搜到 suspend entry 关键字，说明有调用 suspend 流程。

可以根据 log 关键字判断是在哪个环节 Suspend 失败被打回。

搜索 Freezing of tasks failed after 关键字

```
[ 3513.234378] Freezing user space processes ...
[ 3513.240664] PM: Wakeup pending, aborting suspend
[ 3513.240804] Freezing of tasks aborted after 0.006 seconds
[ 3513.240816] OOM killer enabled.
[ 3513.240821] Restarting tasks ...
```

冻结用户空间进程失败，原因是冻结过程中某个唤醒源 Pending，

可以通过 `cat /d/wakeup_sources` 判断最近有哪些唤醒源 Pending。

搜索 PM: dirty and writeback data is 关键字

```
[ 7988.823659] PM: suspend entry (deep)
[ 7988.823737] PM: PM: dirty and writeback data is 332 kB, it's too much for sys_sync, try again!
[ 7988.823742] PM: suspend exit
```

文件系统回写数据过多，需要等待文件系统同步后才能进行深睡操作。

如果连续多次由于该原因导致无法深睡，可能是 log 过多导致，可以关闭 ylog 后重新测试。

搜索 failed to suspend 关键字

```
[ 145.463740] Freezing remaining freezable tasks ... (elapsed 0.004 seconds) done.
[ 145.468370] Suspending console(s) (use no console suspend to debug)
[ 145.471257] dpm run callback(): platform pm suspend+0x0/0x64 returns -22
[ 145.471270] PM: Device sc27xx-fgu failed to suspend: error -22
[ 145.471282] PM: Some devices failed to suspend, or early wake event detected
[ 145.474613] charger-manager charger-manager: Ignoring full-battery soc(state of charge) threshold
as it is not supplied
[ 145.474630] charger-manager charger-manager: Cannot find power supply "hl7028_charger"
```

该问题为充电芯片识别错误，导致 fgu 驱动 suspend 失败。

类似问题可能发生在所有驱动上，可以根据失败的驱动名称，找到对应模块进行下一步分析。

3.2.2 Suspend 流程未被执行

Log 中长时间无法搜到 PM: suspend entry 关键字，说明 Suspend\Resume 流程未被执行，系统持锁。

首先确定上层是否拿锁：

可以通过 Console 输入命令 `dumpsys power`

搜索 Wake Locks: size 关键字

若 size 大于 0 则说明上层持锁，根据关键字后续的 log 得到上层持锁的详细信息。

```
Wake Locks: size=4
PARTIAL_WAKE_LOCK          '*alarm*' ACQ=-18s146ms (uid=1000 pid=990 ws=WorkSource{10164})
PARTIAL_WAKE_LOCK          'AudioIn' ACQ=-8m59s399ms LONG (uid=1041 ws=WorkSource{10170})
PARTIAL_WAKE_LOCK          'deviceidle_going_idle' ACQ=-1m38s22ms LONG (uid=1000 pid=990)
PARTIAL_WAKE_LOCK
'*job*/com.google.android.gm/com.android.mail.job.NotifyDatasetChangedJob$NotifyDatasetChangedJobService' ACQ=-723ms (uid=1000 pid=990 ws=WorkSource{10147})
```

否则为底层持锁：

可以通过 Console 输入命令 `cat /d/wakeup_sources`

该命令将以表格形式打印出所有唤醒源信息

可以通过 `active_since`、`last_change` 等列的数据，判断该唤醒源是否正在 Pending，或是否近期发生过状态改变，从而定位可疑的唤醒源。

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event1	30	30	2	0	0	3882	3488	1969444	0
event0	8	8	0	0	0	1	0	1969258	0
event2	0	0	0	0	0	0	0	0	0
event3	0	0	0	0	0	0	0	0	0
event4	2404	2404	1	0	0	1063	108	1712732	0
event5	0	0	0	0	0	0	0	0	0
wireless	1	1	0	0	0	0	0	1369	0
battery	7	7	0	0	0	67	30	1968006	0
gpio-keys	16	16	3	0	0	11	1	1969439	0
musb-sprd-pd	5	5	0	0	0	2181	804	1967950	0
musb-sprd	2	2	0	0	185828	376505	190676	1967944	0

3.3 异常唤醒分析

如果 AP 睡眠唤醒正常，但从电流图上看存在较多的唤醒，导致功耗偏高，可以搜索中断相关的关键字，确定是什么中断唤醒了系统。

3.3.1 GPIO、EIC 中断唤醒

对于 GPIO、EIC 中断，在其 `resume` 回调中，会检查唤醒源信息，如果为本模块的中断造成的唤醒，将会直接打印唤醒源

可以搜索 `wake up by` 关键字，

音量下键（EIC）

```
[ 307.815030] c2 sprd eic suspend
[ 318.914703] c0 wake up by eic:Volume Down Key id:2
[ 318.923754] c0 sprd_eic_resume
```

Power 键(PMIC EIC)

```
[ 327.010347] c1 sprd pmic eic suspend
[ 331.501478] c0 wake up by pmic eic:Power Key id:1
[ 331.670140] c0 sprd_pmic_eic_resume
```

音量上键（PMIC EIC）

```
[ 1726.352815] c1 sprd pmic eic suspend
[ 1734.300504] c0 wake up by pmic eic:Volume Up Key id:10
[ 1734.458931] c0 sprd_pmic_eic_resume
```

音量下键（GPIO 方式，改为 GPIO 方式进行模拟）

```
[ 647.009723] c2 sprd gpio suspend
[ 659.673561] c0 wake up by gpio:volume down key id:124
[ 659.689389] c0 sprd_gpio_resume
```

3.3.2 其他中断唤醒

可以搜索####--WAKEUP SOURCE 关键字，

```
[ 249.570986] ####--WAKEUP SOURCE
[ 249.597752] ####--ANA (RTC_INT)
[ 277.603984] ####--WAKEUP SOURCE
[ 277.607326] ####--ANA (EIC_INT.PBINT)
[ 365.735243] ####--WAKEUP SOURCE
[ 365.743501] ####--ANA (RTC_INT)
[ 548.549422] ####--WAKEUP SOURCE
[ 548.559708] ####--ANA (RTC_INT)
```

Log 中包含了三次软件通过 alarm timer 设置的 rtc 唤醒，和一次手动按下 power 键引起的 PBINT 唤醒。

可以通过分析哪个中断源唤醒得最为频繁，找到对应模块进行分析。

3.4 睡眠后底电流偏高问题分析

3.4.1 判断系统进入深睡状态

由于整个系统进入深睡，需要依赖所有的子系统进入深睡，依赖条件较多，以下为几种确认系统是否进入深睡的方法。

- 1 量取板子上 Pad_out_chip_sleep 管脚的电平，当整个系统进入深睡时，该管脚会拉高。
- 2 量取板子上 VDDCORE 的电压，当整个系统进入深睡时，VDDCORE 会降压到 0.6V。
- 3 观察系统功耗，如电流平稳，底电流在 10mA 以下，则整个系统已进入深睡。

说明

注：如果外设漏电过大，系统深睡时的电流也可能会高于 10mA，并不能通过功耗反推系统没有进入深睡。

3.4.2 系统未进入深睡时功耗问题分析

首先判断是哪些子系统未进入深睡。

```
[ 208.695359] ####--PMU APB.PWR STATUS0 DBG: 0x07000700
[ 208.695359] ####--PD_AP_SYS_STATE: 0
[ 208.695359] ####--PD_CDMA_SYS_STATE: 0
[ 208.695359] ####--PMU APB.PWR STATUS1 DBG: 0x07070707
[ 208.695359] ####--PMU APB.PWR STATUS2 DBG: 0x07070707
[ 208.695359] ####--PMU APB.PWR STATUS3 DBG: 0x00070707
[ 208.695359] ####--PMU APB.PWR STATUS4 DBG: 0x07000707
[ 208.695359] ####--PD_PUB_SYS_STATE: 0
```

从 AP 侧的 debug log 中，可以读取到其他子系统的睡眠状态。

通常在飞行模式无卡待机的场景，AP 打印 debug log 时，只有 AP 和 PUB 两个子系统仍然处于运行状态。如果有其他的子系统没有睡眠，则优先检查这些子系统是否工作正常。

如果没有 AP 和 PUB 外的子系统在工作，那么需要检查 AP 上依赖的各个模块是否正常关闭。

```
[ 208.695359] ##--AP AHB.AHB EB: 0x00000080  
[ 208.695359] ##--AP APB.APB EB: 0x04028000
```

如有依赖模块的使能位仍为 1，则需要检查相关模块是否正常 Suspend。

3.4.3 系统已进入深睡时功耗问题分析

此时芯片内部的各个子系统已正常关闭，系统进入深睡，PMIC 芯片接收到信号，将其下多数的 LDO、DCDC 关闭或切换到低功耗状态。

这种情况通常是外设没有正常关闭引起的漏电。如 TP、屏幕等没有正常关闭，导致设备漏电。

可以请硬件同事进行电流分解，和平台机器对比各个电源域的功耗，找出功耗偏高的电源域，根据原理图定位该电源域上耗电的模块，从而判断是哪些外设耗电过大。